



FilaCero

Salud sin filas

Guía del código del frontend

Cómo está construido, cómo funciona y cómo extenderlo

Next.js 16 · React 19 · TypeScript · Tailwind CSS 4 · Axios · Zustand

Proyecto integrador · Ingeniería de Software I · Universidad de Cundinamarca

Versión del documento: 28 de septiembre de 2026

Contenido

1. Qué es este frontend	4
1.1 Tecnologías	4
2. Cómo correrlo	4
2.1 Variables de entorno (.env.local)	5
2.2 Scripts de package.json	5
2.3 Primer administrador	5
3. Estructura de carpetas	5
4. Arquitectura por capas (arquitectura limpia)	6
4.1 Ejemplo: qué pasa al pedir un turno	6
4.2 El contenedor (container.ts)	7
5. Autenticación y comunicación con el backend	7
5.1 Inicio de sesión paso a paso	7
5.2 Archivos que participan	7
6. Rutas y pantallas por rol	8
6.1 Vistas del paciente	9
6.2 Vistas del personal y del administrador	12
7. La capa de dominio y aplicación	13
7.1 Entidades (src/core/domain/entities)	13
7.2 Errores (errors.ts)	14
7.3 Puertos y casos de uso	14
8. La capa de presentación	14
8.1 Estructura visual (layout)	14
8.2 Componentes base (components/ui)	14
8.3 Pantallas (components/...)	15
8.4 Hooks (lógica reutilizable)	15
9. Estado global con Zustand	16
10. Diseño: colores y tokens	16
11. Qué funciona con el backend y qué no (todavía)	17
11.1 Funciona de verdad	17
11.2 Datos de ejemplo (src/presentation/mocks/demo.ts)	17
11.3 Botones sin acción (a propósito)	17
11.4 Adaptaciones por límites del backend	17

12. Cómo agregar una funcionalidad nueva	18
12.1 Para una pantalla nueva	18
13. Pruebas con Postman	18
14. Glosario	19

1. Qué es este frontend

El frontend de **FilaCero** es la aplicación web que usan los tres tipos de usuario del sistema: el **paciente**, que pide turnos virtuales y revisa sus medicamentos; el **personal de atención**, que gestiona la cola de un servicio; y el **administrador**, que registra sedes, servicios y usuarios y ve indicadores de la operación.

Está conectado al backend del repositorio **Backend-Duo.exe** (NestJS + Prisma + PostgreSQL). Una regla guía todo el proyecto: **el backend no se modifica**. El frontend se adapta a los endpoints, los datos y las reglas que el backend ya tiene.

1.1 Tecnologías

Tecnología	Versión	Para qué se usa
Next.js (App Router)	16.3.6	Framework de React: rutas por carpetas, páginas en el servidor, Route Handlers (mini-API) y proxy.ts.
React	19.2	Construir la interfaz con componentes.
TypeScript	5	JavaScript con tipos: detecta errores antes de ejecutar.
Tailwind CSS	4	Estilos con clases utilitarias (p. ej. rounded-2xl bg-superficie).
Axios	1.20	Hacer peticiones HTTP al backend (desde el navegador y desde el servidor).
Zustand	5	Estado global pequeño: sesión, turnos del paciente, puesto de atención, menú móvil.
lucide-react	1.48	Iconos (los mismos del diseño de Figma).
@fontsource-variable/inter	5.3	La letra Inter del diseño, incluida en el proyecto (no depende de internet).
server-only	0.0.1	Evita que código de servidor (con el token) termine en el navegador.

Las pantallas del paciente siguen el archivo de Figma **"FilaCero · Frontend simplificado"**, en versión de escritorio y de celular.

2. Cómo correrlo

Se necesitan **dos terminales abiertas al mismo tiempo**: una para el backend (puerto 3000) y otra para el frontend (puerto 3001). También Node.js 22 o superior y Docker Desktop para la base de datos.

Terminal 1 · backend (carpeta Backend-Duo.exe)

```
npm install
Copy-Item .env.example .env      # solo la primera vez
docker compose up -d             # enciende PostgreSQL
npx prisma migrate dev           # crea las tablas (primera vez)
npm run start:dev                 # espera "Nest application successfully started"
```

Terminal 2 · frontend (carpeta Frontend)

```
npm install
Copy-Item .env.example .env.local # solo la primera vez
npm run dev                       # espera "Ready" y abre http://localhost:3001
```

2.1 Variables de entorno (.env.local)

Variable	Valor por defecto	Qué es
API_URL	http://localhost:3000	Dirección del backend. Solo la usa el servidor de Next.js; nunca llega al navegador.
AUTH_COOKIE_NAME	filacero_token	Nombre de la cookie donde se guarda el token de sesión.

2.2 Scripts de package.json

Comando	Qué hace
npm run dev	Modo desarrollo en el puerto 3001; recarga al guardar cambios.
npm run build	Compila la versión de producción (también revisa tipos).
npm start	Corre la versión compilada en el puerto 3001.
npm run lint	Revisa el estilo y errores comunes con ESLint.
npm run typecheck	Revisa los tipos de TypeScript sin compilar.

2.3 Primer administrador

El registro público siempre crea usuarios con rol **PACIENTE**, y crear usuarios de otro rol exige ser **ADMIN**. Por eso el primer administrador se crea así: regístrate en la página, luego en la carpeta del backend corre `npx prisma studio`, abre la tabla **Usuario**, cambia tu **rol** a **ADMIN** y guarda. Desde ahí, en **Administración** → **Servicios**, se crean sedes, servicios y personal de atención.

Si el navegador dice “localhost rechazó la conexión”

No hay ningún servidor corriendo en ese puerto: la terminal de `npm run dev` se cerró, falló o se corrió en otra carpeta. Si la página se ve antigua, detén el servidor, borra la carpeta `.next` y vuelve a correr `npm run dev`.

3. Estructura de carpetas

El proyecto separa las **rutras** (carpeta `app/`, que exige Next.js) del **código de la aplicación** (carpeta `src/`, organizada por capas).

```

Frontend/
├── app/
│   ├── layout.tsx          ← RUTAS (cada carpeta = una URL)
│   ├── globals.css         ← HTML base, letra Inter y estilos globales
│   ├── page.tsx            ← colores y tokens del diseño
│   ├── (auth)/login, registro ← portada pública (/)
│   ├── (app)/              ← páginas públicas
│   │   ├── layout.tsx      ← páginas que exigen sesión
│   │   ├── inicio, turnos, turnos/solicitar, medicamentos, historial (paciente)
│   │   ├── fila, fila/atendidos, fila/historial (personal)
│   │   └── admin, admin/servicios, admin/inventario, admin/reportes (admin)
│   └── api/                ← mini-API del frontend (Route Handlers)
│       ├── auth/login, register, logout, session
│       └── bff/[...path]    ← puente hacia el backend
├── proxy.ts                ← protege las rutas según la sesión y el rol
├── src/
│   ├── config/env.ts       ← lee las variables de entorno
│   ├── core/
│   │   ├── domain/entities ← DOMINIO: reglas del negocio, sin frameworks
│   │   │   ├── Turno, Usuario, Rol, Sesión, Sede, Servicio
│   │   ├── domain/ports    ← interfaces (qué operaciones existen)
│   │   └── application/use-cases ← casos de uso (validar y ejecutar)
│   ├── infrastructure/
│   │   ├── http/           ← cómo se habla con el backend
│   │   │   ├── cliente Axios y traducción de errores
│   │   ├── mappers/        ← convierte JSON del backend a objetos del dominio
│   │   ├── repositories/   ← implementan los puertos con Axios
│   │   └── container.ts     ← conectan todo: aquí se crean los casos de uso
│   ├── server/             ← código SOLO de servidor (token, sesión)
│   └── presentation/
│       ├── components/     ← todo lo visual
│       ├── hooks/          ← pantallas y piezas de interfaz
│       ├── stores/         ← lógica reutilizable de React
│       ├── mocks/demo.ts   ← estado global (Zustand)
│       ├── lib/cn.ts       ← datos de ejemplo
│       └── lib/cn.ts       ← utilidades (clases, fechas)
├── postman/                ← colección y entorno para probar la API
└── package.json, tsconfig.json, .env.example, README.md

```

4. Arquitectura por capas (arquitectura limpia)

La idea central es que **las reglas del negocio no dependan de cómo se muestran ni de cómo se consiguen los datos**. Cada capa solo conoce a la que está “más adentro”:

Capa	Carpeta	Responsabilidad	Puede usar
Dominio	src/core/domain	Qué es un turno, qué estados tiene, qué transiciones son válidas, cómo se calcula el código corto.	Nada externo
Aplicación	src/core/application	Casos de uso: validar los datos y pedir la operación a un puerto.	Dominio
Infraestructura	src/infrastructure, src/server	Hablar con el backend (Axios), traducir JSON, manejar el token.	Dominio, aplicación
Presentación	src/presentation, app/	Pantallas, formularios, estado de la interfaz.	Todas

4.1 Ejemplo: qué pasa al pedir un turno

- La pantalla (solicitar-turno.tsx) llama a `casosDeUso.solicitarTurno.ejecutar(servicioId)`.

- El caso de uso (SolicitarTurnoUseCase) valida que haya un servicio y llama al puerto TurnoRepository.solicitar.
- La implementación (HttpTurnoRepository) hace POST /api/bff/turnos con Axios.
- El BFF de Next.js agrega el token y reenvía a POST http://localhost:3000/turnos.
- La respuesta JSON se convierte en un objeto Turno con TurnoMapper y vuelve a la pantalla, que lo guarda en Zustand y navega al seguimiento.

¿Por qué tanto orden?

Si mañana el backend cambia una URL, solo se toca el repositorio. Si cambia el diseño, solo la presentación. Y las reglas (por ejemplo, “un turno ATENDIDO no se puede cancelar”) viven en un solo lugar: core/domain/entities/turno.ts.

4.2 El contenedor (container.ts)

Es el único lugar donde se decide qué implementación usa cada puerto. Exporta el objeto casosDeUso, que las pantallas importan para hacer cualquier operación:

```
export const casosDeUso = {
  iniciarSesion, registrarse, cerrarSesion, // autenticación
  listarEntidades, listarServicios, crearEntidad, crearServicio,
  solicitarTurno, cancelarTurno, avanzarTurno, consultarFila,
  listarUsuarios, crearUsuario,
};
```

5. Autenticación y comunicación con el backend

El backend entrega un **token JWT** al iniciar sesión y exige recibirlo en cada petición en la cabecera `Authorization: Bearer <token>`. Guardar ese token en JavaScript sería inseguro (un script malicioso podría robarlo), así que el frontend usa un patrón llamado **BFF** (Backend For Frontend):

Navegador —(cookie httpOnly)→ Next.js /api/* —(Authorization: Bearer)→ Backend NestJS :3000

5.1 Inicio de sesión paso a paso

- El formulario envía correo y contraseña a POST /api/auth/login (un Route Handler de Next.js).
- El Route Handler llama a POST /auth/login del backend y recibe { accessToken }.
- Guarda el token en la cookie filacero_token con **httpOnly** (JavaScript no la puede leer), **SameSite=Lax** y **Secure** en producción. Dura lo mismo que el token (1 día).
- Consulta GET /usuarios/:id para obtener el nombre y responde al navegador solo con la **sesión** (id, nombre, correo, rol), nunca con el token.

5.2 Archivos que participan

Archivo	Qué hace
app/api/auth/_shared.ts	Lógica común de login y registro: llama al backend, guarda la cookie y arma la sesión.
app/api/auth/login · register	Rutas POST que usan la lógica común.

Archivo	Qué hace
app/api/auth/logout	Borra la cookie (el backend no tiene logout: el JWT no guarda estado).
app/api/auth/session	Devuelve la sesión actual o 401 si no hay cookie válida.
app/api/bff/[...path]/route.ts	Puente genérico: toma cualquier /api/bff/X y lo reenvía a {API_URL}/X con el token. Solo permite entidades-medicas, servicios, turnos y usuarios; rechaza mutaciones de otro origen (protección CSRF); si el backend responde 401, borra la cookie.
src/server/backend-client.ts	Cliente Axios de servidor hacia el backend; convierte la cookie en Bearer.
src/server/session.ts	Lee la cookie, construye la sesión y define las opciones de la cookie.
src/server/jwt.ts	Lee (sin verificar) los datos del token: id, correo, rol y vencimiento. La verificación real la hace el backend.
src/infrastructure/http/internal-client.ts	Cliente Axios del navegador (baseURL /api). Si recibe 401 fuera de /auth, avisa para cerrar la sesión y enviar a /login.
src/infrastructure/http/normalizar-error.ts	Convierte cualquier error (incluidos los de NestJS con message en lista) en un AppError con mensajes en español.
proxy.ts	Antes de mostrar una página revisa la cookie: sin sesión envía a /login; con un rol sin permiso envía a su portada; y manda a cada rol a su inicio (paciente /inicio, personal /fila, admin /admin).

Seguridad en resumen

El token nunca llega al JavaScript del navegador; las mutaciones solo se aceptan desde el mismo origen; el frontend no conoce JWT_SECRET y no lo necesita, porque el backend valida el token y los permisos (JwtAuthGuard y RolesGuard) en cada petición.

6. Rutas y pantallas por rol

Ruta	Rol	Pantalla	Endpoints del backend
/	Público	Portada con botones de registro e inicio de sesión.	—
/login · /registro	Público	Formularios de acceso.	POST /auth/login · /auth/register
/inicio	Paciente	Saludo, turno en curso, reserva lista y accesos rápidos.	(usa los turnos guardados)
/turnos/solicitar	Paciente	Solicitud en 3 pasos: sede y servicio → datos → confirmación.	GET /entidades-medicas, GET /servicios?entidadId, POST /turnos
/turnos	Paciente	Mi turno: código, posición, espera estimada, sede, cancelar.	PATCH /turnos/:id/cancelar
/medicamentos	Paciente	Fórmula activa, reserva y disponibilidad por sede.	Datos de ejemplo
/historial	Paciente	Tabla de todos sus turnos.	(turnos guardados)
/fila	Personal / Admin	Cola de atención: indicadores, atendiendo ahora, siguiente, cola actual.	GET /turnos/servicio/:id, PATCH .../avanzar, PATCH .../cancelar

Ruta	Rol	Pantalla	Endpoints del backend
/fila/atendidos · /fila/historial	Personal / Admin	Turnos finalizados o todos los del servicio.	GET /turnos/servicio/:id
/admin	Admin	Resumen operativo con indicadores reales y gráfico de demanda por hora.	GET /entidades-medicas, /servicios, /turnos/servicio/:id
/admin/servicios	Admin	Crear sedes, servicios y usuarios; listas.	POST /entidades-medicas, /servicios, /usuarios; GET /usuarios
/admin/inventario · /admin/reportes	Admin	Pantallas de diseño con datos de ejemplo.	—

Grupos de rutas. Las carpetas entre paréntesis, como (app) y (auth), no aparecen en la URL: sirven para compartir un layout.tsx. El de (app) confirma la sesión en el servidor, pinta la barra lateral y le pasa la sesión a Zustand.

6.1 Vistas del paciente

The screenshot displays the FilaCero patient portal. The header includes the FilaCero logo and the patient's name, Laura Rodríguez. The main content area is titled 'Tu salud, sin filas innecesarias' and shows the patient's current appointment status. The appointment is at Centro de Salud Chapinero, and the patient is currently in the queue, with a position of 4. The interface also shows a list of upcoming appointments, including one for Losartán 50 mg y Acetaminofén 500 mg. The sidebar on the left contains navigation links for 'Inicio', 'Mis turnos', 'Medicamentos', and 'Historial'. The bottom of the page features a section for 'Accesos rápidos' (Quick access) with buttons for 'Solicitar un turno', 'Consultar medicamentos', and 'Agendar una cita'.

Inicio del paciente (escritorio).

FilaCero

Salud sin filas

PERFIL ACTIVO

Paciente

Inicio

Mis turnos

Medicamentos

Historial

¿Necesitas ayuda?

Línea gratuita 01 8000 123 456

FilaCero no diagnostica ni reemplaza la atención médica.

TURNOS VIRTUALES

Solicitar un turno

Selecciona dónde y para qué trámite necesitas atención.

1

Sede y servicio

2

Tus datos

3

Confirmación

1. Selecciona una sede

Elige la sede de tu red de salud donde quieres ser atendido.

Ubicación activa

Ciudad

Bogotá D. C.

Zona o barrio

Todas las zonas

Farmacia Central Calle 80

Sede de la red de salud

Centro de Salud Chapinero

Sede de la red de salud

IPS Galerias

Sede de la red de salud

2. Elige el servicio

Podrás realizar un trámite por turno.

Dispensación de medicamentos

Disponible en esta sede

Asignación de citas

Disponible en esta sede

Autorizaciones

Disponible en esta sede

Información y orientación

Disponible en esta sede

Resumen

Revisa tu selección antes de continuar.

SEDE

Centro de Salud Chapinero

SERVICIO

Asignación de citas

Espera estimada

Al confirmar

Continuar

Tus datos se usan únicamente para gestionar la atención solicitada.

Atención prioritaria

Si eres adulto mayor, gestante o tienes movilidad reducida, indícalo en el siguiente paso.

Solicitud de turno, paso 1 (escritorio).

FilaCero

Salud sin filas

PERFIL ACTIVO

Paciente

Inicio

Mis turnos

Medicamentos

Historial

¿Necesitas ayuda?

Línea gratuita 01 8000 123 456

FilaCero no diagnostica ni reemplaza la atención médica.

SEGUIMIENTO EN TIEMPO REAL

Mi turno

Consulta el estado de tu turno y cuánto falta para tu atención.

✓

¡Turno asignado!

Guarda tu código T-6EB557. Te avisaremos cuando se acerque tu llamado.

Tu turno está en camino

Centro de Salud Chapinero · Asignación de citas

Actualizado hace un momento

Mantente atento

Dirígete a la sede cuando falten 2 personas. Debes presentar tu documento y fórmula médica vigente.

TU CÓDIGO

En espera

T-6EB557

Laura Rodríguez

l*****@correo.co

Turno registrado · actualizado hace un momento

Estado de la fila

El tiempo puede variar según la duración de cada atención.

Tu posición

4

en la fila

Antes que tú

3

personas

Espera estimada

18

minutos

Ingreso a la fila

25% de cercanía

Atención

SOLICITADO

Asignación de citas

1:05 a. m.

Información de la sede

Centro de Salud Chapinero

Asignación de citas · Entrada principal

Cómo llegar

Notificaciones

Correo a l*****@correo.co

Activas

Cancelar turno

Ver historial de turnos

Mi turno (escritorio).



Inicio del paciente (celular), con la barra de navegación inferior.

6.2 Vistas del personal y del administrador

FilaCero

PERFIL ACTIVO

Cola de turnos

Turnos atendidos

Historial

¿Necesitas ayuda?

FARMACIA CENTRAL CALLE 80

Cola de atención

En servicio

Camila Sánchez

Turnos por atender

Puesto de atención

En espera

Atendidos hoy

Espera promedio

Atención promedio

Atendiendo ahora

Siguiente en fila

Atención prioritaria

Cola actual

Personal de atención

Turno de la mañana - 7:00 a. m. a 1:00 p. m.

Pausar ventanilla

Cerrar turno

Sede

Servicio

Farmacia Central Calle 80

Dispensación de medicamentos

6

1

0 min

Se calculará con los tiempos de atención

CONFIRMA EL RESULTADO PARA AVANZAR LA FILA.

En atención - 1

TURNOS ACTUALES

Paciente C55D2

Dispensación de medicamentos

LLEGADA

1:05 a. m.

✓ Marcar como atendido

No se presentó

Transferir

Llamar al siguiente

El turno pasará a "En atención".

Adultos mayores, gestantes y personas con movilidad reducida se atienden según el protocolo de prioridad de la sede.

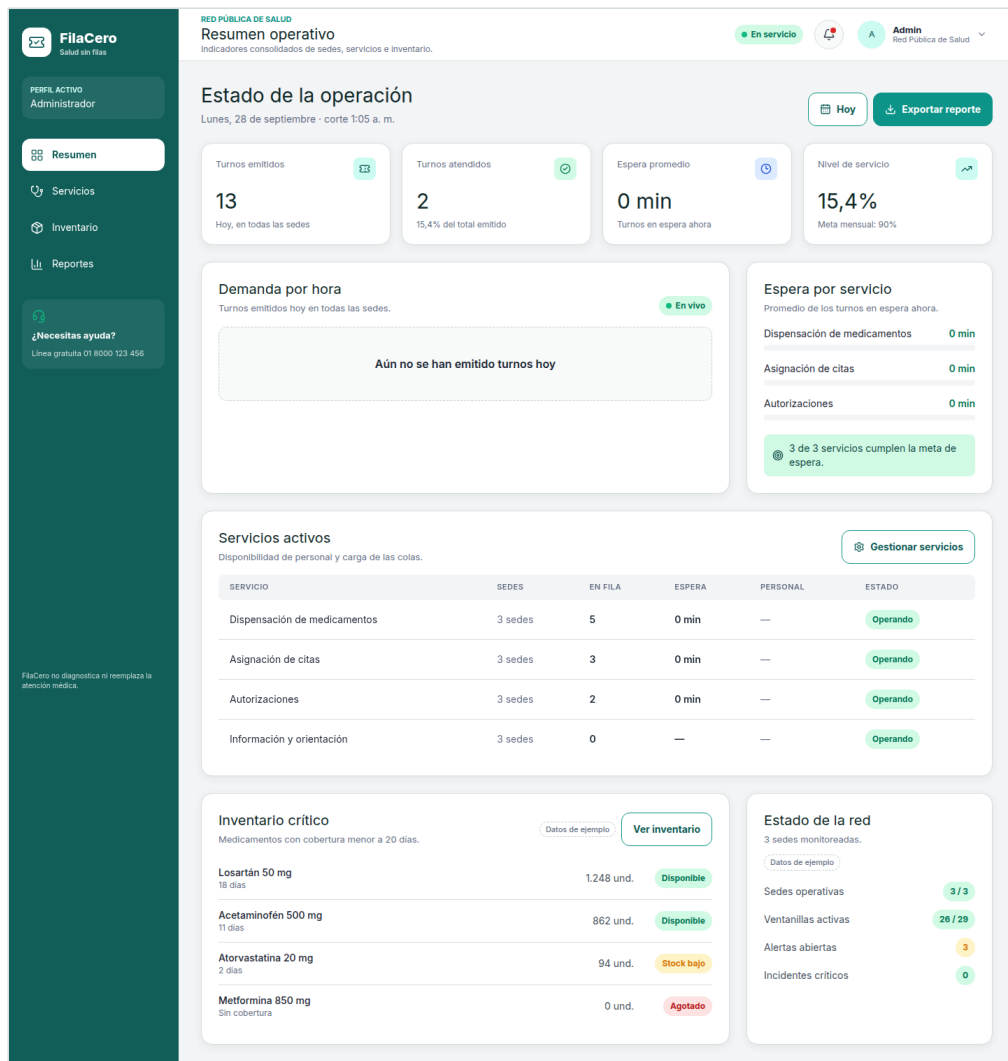
6 personas esperan atención - orden de llegada - actualizado 1:05 a. m.

Auto (10 s)

Actualizar

TURNOS	PACIENTE	SERVICIO	LLEGADA	ESPERA	PRIORIDAD
T-B8854C	Paciente 099D85	Dispensación de medicamentos	1:05 a. m.	0 min	General
T-47FD02	Paciente 49A596	Dispensación de medicamentos	1:05 a. m.	0 min	General
T-3A3BF2	Paciente AAD8F5	Dispensación de medicamentos	1:05 a. m.	0 min	General
T-6090CC	Paciente 510CF0	Dispensación de medicamentos	1:05 a. m.	0 min	General
T-45CF70	Paciente 366DFB	Dispensación de medicamentos	1:05 a. m.	0 min	General
T-AED404	Paciente 218309	Dispensación de medicamentos	1:05 a. m.	0 min	General

Cola de atención del personal.



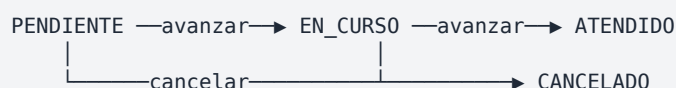
Resumen operativo del administrador.

7. La capa de dominio y aplicación

7.1 Entidades (src/core/domain/entities)

Archivo	Contenido
rol.ts	Los roles del backend: PACIENTE, FUNCIONARIO, ADMIN, sus etiquetas y esStaff().
sesion.ts	JwtPayload (lo que trae el token) y Sesion (lo que usa la interfaz).
usuario.ts	Usuario, NuevoUsuario, DatosRegistro y Credenciales, espejo de los DTO del backend.
catalogo.ts	EntidadMedica (sede) y Servicio.
turno.ts	Turno y EstadoTurno (PENDIENTE, EN_CURSO, ATENDIDO, CANCELADO); reglas puedeAvanzar, puedeCancelar, estaActivo; codigoCorto() (T-XXXXXX a partir del UUID) y estimarEspera() con el supuesto MINUTOS_POR_TURNOS = 6.

Máquina de estados del turno (copiada del backend)



7.2 Errores (errors.ts)

AppError guarda una lista de mensajes y el código HTTP. Todas las pantallas muestran los errores con el mismo componente (ListaErrores), así los mensajes de validación del backend llegan tal cual al usuario.

7.3 Puertos y casos de uso

ports/index.ts define las interfaces AuthRepository, CatalogoRepository, TurnoRepository y UsuarioRepository. Los casos de uso (una clase con un método ejecutar()) validan antes de llamar al backend, con las mismas reglas de sus DTO:

Caso de uso	Validación / lógica
IniciarSesion · Registrarse	Correo válido; contraseña de mínimo 6 caracteres; nombre obligatorio. Normaliza el correo a minúsculas.
SolicitarTurno	Exige un servicio seleccionado.
CancelarTurno · AvanzarTurno	Rechazan transiciones que el backend no permite.
ConsultarFila	Trae la fila del servicio y calcula el conteo por estado y el siguiente turno.
CrearEntidad · CrearServicio · CrearUsuario	Nombres obligatorios; los datos del usuario se validan como en el registro.

8. La capa de presentación

8.1 Estructura visual (layout)

Archivo	Qué hace
layout/app-shell.tsx	Esqueleto de la zona privada. En escritorio: barra lateral de 244 px. En celular: barra de navegación inferior y la hoja de Perfil (Historial, ayuda, Cerrar sesión y Eliminar cuenta).
layout/pagina.tsx	Cada página se envuelve en <Pagina>, que pinta la cabecera: blanca con título y menú de usuario en escritorio, cerceta con marca y avatar en celular. Contiene el menú de usuario con Cerrar sesión y el botón Eliminar cuenta (sin acción).
layout/navegacion.ts	Qué opciones de menú ve cada rol, sus iconos y la portada de cada uno (INICIO_POR_ROL).
layout/logo.tsx	La marca FilaCero.
layout/sesion-hydrator.tsx	Pasa la sesión del servidor a Zustand y define qué hacer si la sesión expira.

8.2 Componentes base (components/ui)

Componente	Uso
Button · BotonEnlace · claseBoton	Botones con variantes: primario (esmeralda), estructura (cerceta), secundario (borde), peligro (rojo) y fantasma. BotonEnlace es un enlace con aspecto de botón.
Card · CardTitulo · EncabezadoPagina	Tarjetas blancas con borde y sombra; títulos de tarjeta y de página.

Componente	Uso
Indicador · Pildora · MarcaDemo	Tarjeta de indicador (KPI), etiqueta de estado redondeada y la marca “Datos de ejemplo”.
Alert · ListaErrores	Avisos de información, éxito, advertencia o error.
Input · Select	Campos de formulario con etiqueta, ayuda y error accesibles.
BadgeEstadoTurno	Etiqueta del estado de un turno.
EstadoVacio · Spinner	Mensaje cuando no hay datos y animación de carga.

8.3 Pantallas (components/...)

Carpeta / archivo	Pantalla
paciente/dashboard-paciente.tsx	Inicio del paciente (versión de escritorio y de celular).
turnos/solicitar-turno.tsx	Solicitud en 3 pasos. Elige el icono del servicio según su nombre.
turnos/mis-turnos.tsx	Mi turno (seguimiento) y la lista de otros turnos activos.
paciente/medicamentos-paciente.tsx	Fórmula, reserva y disponibilidad (datos de ejemplo).
paciente/historial-paciente.tsx	Historial del paciente.
fila/panel-fila.tsx · selector-puesto.tsx · tabla-turnos-servicio.tsx	Cola de atención, selección de sede/servicio y tablas de atendidos/historial.
admin/resumen-operativo.tsx · grafico-demanda.tsx	Resumen del admin y gráfico de barras de demanda por hora.
admin/panel-admin.tsx	Formularios de sedes, servicios y usuarios.
auth/login-form.tsx · registro-form.tsx	Formularios de acceso.

8.4 Hooks (lógica reutilizable)

Hook	Qué hace
useCatalogo(entidadId)	Carga las sedes y, al elegir una, sus servicios.
useCola()	Estado del puesto de atención: sede, servicio, fila, actualización cada 10 s y nombres (si es admin).
useResumenOperativo()	Calcula los indicadores del admin a partir de todos los turnos (se refresca cada 30 s).
useMontado() · useAhora()	Evitar diferencias entre servidor y navegador, y actualizar textos como “hace 2 min”.

9. Estado global con Zustand

Zustand guarda datos que varias pantallas necesitan al mismo tiempo, sin pasarlos componente por componente.

Store	Qué guarda	Dónde se guarda
sesion.store.ts	El usuario autenticado (nombre, correo, rol).	Solo en memoria. Nunca guarda el token.
turnos.store.ts	Los turnos que ha pedido cada paciente, con el nombre de sede y servicio.	localStorage (clave filacero-turnos)
fila.store.ts	La sede y el servicio que eligió el personal de atención.	sessionStorage (dura mientras la pestaña esté abierta)
ui.store.ts	Si la hoja de Perfil del celular está abierta.	Solo en memoria

¿Por qué los turnos del paciente se guardan en el navegador?

El backend no tiene un endpoint para que el paciente consulte sus propios turnos: la posición solo llega cuando se crea o se cancela un turno. Por eso el frontend guarda esa respuesta. Consecuencia: la posición del paciente no se actualiza sola y los turnos solo se ven en el mismo navegador.

10. Diseño: colores y tokens

Los colores están definidos una sola vez en `app/globals.css` (bloque `@theme`) y se usan como clases de Tailwind, por ejemplo `bg-cerceta-oscuro` o `text-tinta-tenue`. Para cambiar un color en toda la aplicación basta con editarlo ahí.

	Token	Hex	Uso
	neutro	#f3f4f6	Fondo de la página (60 %)
	cerceta-oscuro	#115e59	Barra lateral, cabecera móvil, tarjeta del código
	cerceta	#0d9488	Estructura, bordes activos, antetítulos (30 %)
	esmeralda	#10b981	Botón principal y estados de farmacia (10 %)
	aqua	#ccfbf1	Avisos informativos, selección
	menta	#d1fae5	Estados positivos, espera estimada
	ambar-fondo	#fef3c7	Advertencias
	titulo	#102a2a	Títulos
	tinta	#1f2937	Texto
	tinta-tenue	#667085	Texto secundario
	borde	#d6dee0	Bordes

Otros valores del diseño: letra **Inter**; tarjetas con radio de 16 px y sombra suave; botones de 48 px de alto y radio de 12 px; en escritorio, barra lateral de 244 px y cabecera de 76 px; en celular, barra inferior de 74 px.

Accesibilidad

Hay un enlace para saltar al contenido, foco visible con el teclado, etiquetas en todos los campos, avisos anunciados a lectores de pantalla y tablas con descripción. Una nota: el botón verde con texto blanco (igual a Figma) tiene poco contraste (2.5:1). Si se necesita cumplir WCAG AA, se cambia `text-white` por `text-tinta` en la variante primario de `button.tsx`.

11. Qué funciona con el backend y qué no (todavía)

11.1 Funciona de verdad

- Registro, inicio y cierre de sesión, con protección de rutas por rol.
- Paciente: ver sedes y servicios reales, pedir turno, ver su código y posición, cancelar.
- Personal: ver la cola de un servicio, llamar al siguiente, marcar atendido, marcar “no se presentó” (cancela), historial.
- Admin: crear sedes, servicios y usuarios de cualquier rol; indicadores calculados con los turnos reales.

11.2 Datos de ejemplo (`src/presentation/mocks/demo.ts`)

El backend no tiene fórmulas, medicamentos, inventario, reservas, citas ni ventanillas. Esas secciones del diseño se llenan con datos fijos de este archivo. Cuando el backend los tenga, se reemplaza este archivo por un repositorio real en `src/infrastructure/repositories`.

11.3 Botones sin acción (a propósito)

Eliminar cuenta · Cómo llegar · Agendar una cita · Reservar medicamentos · Cambiar sede · Buscar otra sede · filtros de Ciudad y Zona · Pausar ventanilla · Cerrar turno · Transferir · Hoy · Exportar reporte · Generar reporte · Registrar ingreso · campana de notificaciones.

11.4 Adaptaciones por límites del backend

Limitación del backend	Cómo lo resuelve el frontend
No genera un código de turno (A-142).	Muestra T-XXXXXX, tomado del identificador del turno.
No hay endpoint para “mis turnos”.	Guarda los turnos del paciente en el navegador (Zustand).
No guarda tiempos de atención.	Espera estimada = (posición – 1) × 6 minutos (supuesto).
No hay espera por sede ni total de la fila.	Muestra “Al confirmar” y “Posición N en la fila”.
El personal no puede consultar usuarios.	Ve “Paciente XXXXXX”; el admin sí ve los nombres.
No hay tiempo real (WebSockets).	La cola se consulta cada 10 s y el resumen cada 30 s.
No hay endpoint para borrar la cuenta propia.	El botón Eliminar cuenta está, pero sin acción.

12. Cómo agregar una funcionalidad nueva

Ejemplo: el backend agrega DELETE /usuarios/:id y se quiere activar **Eliminar cuenta**. Los pasos siguen las capas:

#	Dónde	Qué hacer
1	app/api/bff/[...path]/route.ts	Verificar que el recurso esté permitido (usuarios ya lo está) y que DELETE esté exportado (ya lo está).
2	core/domain/ports/index.ts	Agregar eliminar(id: string): Promise<void> a UsuarioRepository.
3	infrastructure/repositories/usuario.repository.ts	Implementarlo: internalApi.delete(`/bff/usuarios/\${id}`).
4	core/application/use-cases	Crear EliminarCuentaUseCase (por ejemplo, exigir confirmación).
5	infrastructure/container.ts	Registrar eliminarCuenta: new EliminarCuentaUseCase(usuarioRepo).
6	components/layout/pagina.tsx	En BotonEliminarCuenta: pedir confirmación, llamar al caso de uso, luego cerrar sesión y enviar a /login.
7	Probar	npm run typecheck, npm run lint y probar en el navegador. Agregar la petición a la colección de Postman.

12.1 Para una pantalla nueva

- Crear la carpeta de la ruta en app/(app)/nombre/page.tsx y envolver el contenido en <Pagina barra={{...}}>.
- Poner la pantalla en src/presentation/components/... usando los componentes de ui/.
- Agregarla al menú en navegacion.ts y, si es privada, a RUTAS_POR_ROL y al matcher de proxy.ts.

13. Pruebas con Postman

En la carpeta postman/ hay una colección y un entorno (FilaCero – Local). Se importan en Postman con **Import**. La colección tiene dos carpetas:

- **1. Backend NestJS (Bearer):** prueba el backend directo; el login guarda el token en la variable {{token}}.
- **2. Frontend BFF (cookie):** las mismas operaciones pasando por el frontend en el puerto 3001; verifica que la cookie sea httpOnly y que el token no aparezca en la respuesta.

Orden sugerido: Login como admin → Crear sede → Crear servicio → Solicitar turno → Fila → Avanzar.

14. Glosario

Término	Significado
App Router	Sistema de rutas de Next.js: cada carpeta dentro de app/ es una URL.
Route Handler	Archivo route.ts que responde peticiones HTTP; funciona como una mini-API dentro del frontend.
BFF	Backend For Frontend: capa intermedia que protege el token y reenvía las peticiones al backend real.
JWT	Token firmado que prueba quién es el usuario y qué rol tiene.
Cookie httpOnly	Cookie que el navegador envía automáticamente pero que JavaScript no puede leer.
proxy.ts	Código que corre antes de cada página (en versiones anteriores de Next.js se llamaba middleware).
Server / Client Component	Componentes que se ejecutan en el servidor o en el navegador (los segundos llevan "use client").
Store (Zustand)	Contenedor de estado global al que cualquier componente se puede suscribir.
Puerto / Repositorio	Interfaz que define una operación, y la clase que la implementa con HTTP.
Caso de uso	Clase con un método ejecutar() que representa una acción del usuario.
Token de diseño	Variable con nombre (p. ej. cerceta-oscuro) que guarda un color o medida del diseño.